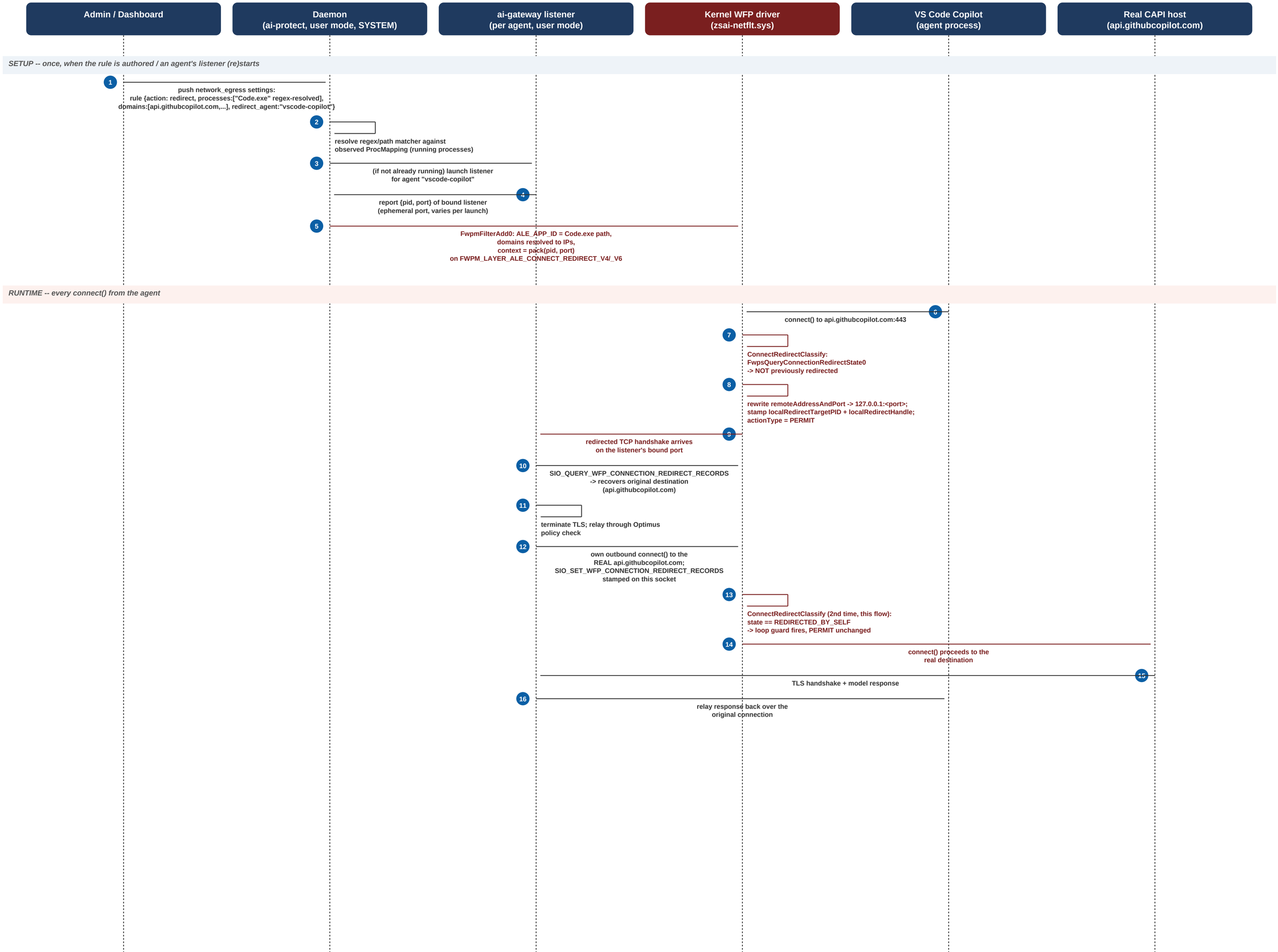

Kernel-mode steps in red. Numbered badges link to the itemization pages; click a number there to jump back here.

Sequence Diagram Itemization -- VS Code Copilot WFP Redirect

Cross-referenced to the numbered diagram on the previous page. Click a step number below to jump to its arrow on the diagram; click a numbered badge on the diagram to jump back here.

Legend -- VERIFIED: real, built, confirmed in this session. PARTIAL: real code exists but is narrower/different than the diagram's simplified depiction. NOT IMPLEMENTED: confirmed absent by direct source search.

1. Admin/Dashboard -> Daemon: push network_egress settings

VERIFIED

Caller: external control plane (not in this repo)

Lands in: apply_net_egress -- src/subscribers/settings_apply.rs:2498, reached via the match settings.get("network_egress") dispatch at settings_apply.rs:627

Before the call: settings_store's live NetEgressConfig holds whatever was last applied (default: disabled, no rules).

After the call: payload deserializes into NetEgressConfig -- now including RuleAction::Redirect / EgressRule::redirect_agent -- and is written via settings_store::update_net_egress. Takes effect on the service loop's NEXT tick, not immediately.

2. Daemon (self): resolve process matcher against observed processes

VERIFIED

Caller: the service loop in src/services/net_egress/mod.rs, via reconcile::compute_redirect_filters(&cfg, &dns, &procs, &listeners)

Lands in: compute_redirect_filters -> config::process_matches -- src/services/net_egress/reconcile.rs / config.rs

Before the call: the rule already passed action == Redirect and constrains_scope(). In the real code (unlike the diagram's simplified left-to-right order) the redirect_agent -> ListenerMapping lookup already happened before this step, as a short-circuit.

After the call: zero or more concrete image paths collected; empty means the function continues to the next rule -- it never widens to "any process".

3. Daemon -> ai-gateway: launch listener for the agent

PARTIAL

Caller: subscribers::ai_broker_launch's settings-driven spawn logic -- src/subscribers/ai_broker_launch.rs

Lands in: no single function call crosses this boundary. This subscriber spawns the whole zscaler-ai-gateway PROCESS and writes .broker.zip; which per-agent listener actually binds is ai-gateway's own internal decision (agent-manager's Spec::listener_transport -> listener/src/kinds/https.rs::bind), in a separate binary, reached only through that file handoff.

Before the call: .broker.zip holds the current discovered-agent list; the gateway process may or may not already be running.

After the call: if not running, the child process spawns; once up, it reads .broker.zip and binds a listener per agent that needs one.

4. ai-gateway -> Daemon: report {pid, port} of the bound listener

NOT IMPLEMENTED

Caller: does not exist

Lands in: would land in correlate::Engine::set_listener_mappings -- src/services/net_egress/correlate.rs -- never called by a real producer today

Before the call: N/A.

After the call: N/A -- listener_mappings() returns empty unless the test-stub env var (ZSAI_NETFLT_TEST_LISTENERS) is set. This is the single hard blocker on the whole daemon-driven pipeline.

5. Daemon -> WFP engine: FwpmFilterAdd0

VERIFIED

Caller: Enforcer::add_redirect_filter, called from Enforcer::reconcile_redirects, called from the service loop -- src/services/net_egress/enforcer.rs

Lands in: the OS's WFP engine, not the driver directly -- it records the filter and its action.calloutKey = ZSAI_NETFLT_CALLOUT_REDIRECT_V4, a GUID the driver registered at load (RegisterCallouts, driver/zsai-netflt/zsai-netflt.c). The driver's own code is referenced, not invoked, at add time.

Before the call: compute_redirect_filters produced a RedirectFilterSpec that reconcile_redirects found missing from installed_redirects.

After the call: the filter is live in the kernel engine (inert until a real connect() matches it); its id is recorded and IdChange::Added published.

6. VS Code Copilot -> kernel: connect()

VERIFIED

Caller: VS Code Copilot's own process (external)

Lands in: ConnectRedirectClassify -- driver/zsai-netflt/zsai-netflt.c (invoked by the OS, not called directly)

Before the call: the filter from step 5 is live; VS Code has resolved DNS and is initiating a TCP connect.

After the call: classify runs synchronously in-kernel (steps 7-8) before the syscall returns to VS Code.

7. Kernel (self): FwpsQueryConnectionRedirectState0 -- loop-guard check #1

VERIFIED

Caller: same function, ConnectRedirectClassify

Lands in: same function, ConnectRedirectClassify

Before the call: actionType already defaulted PERMIT; g_RedirectHandle non-null; context unpacked to a nonzero {pid, port}.

After the call: state is NOT_REDIRECTED (first pass) -> proceeds to step 8 instead of returning early.

8. Kernel (self): rewrite address, stamp PID/handle, actionType=PERMIT

VERIFIED

Caller: same function, via FwpsAcquireClassifyHandle0 / FwpsAcquireWritableLayerDataPointer0 / FwpsApplyModifiedLayerData0

Lands in: same function

Before the call: loop guard passed.

After the call: FWPS_CONNECT_REQUEST0.remoteAddressAndPort is now 127.0.0.1:<port>; the TCP SYN that actually leaves the stack targets the listener, not api.githubcopilot.com.

9. Kernel -> ai-gateway: redirected handshake arrives

VERIFIED

Caller: not a function call -- the OS TCP stack completing the rewritten handshake

Lands in: https::serve -> server::serve_with_prepare's accept loop -> sniff / sniff_inner -- ai-gateway/listener/src/kinds/https.rs:138

Before the call: bind (https.rs:53) already bound this port in step 3.

After the call: a new TcpStream is accepted and handed to sniff_inner to classify ClientHello-vs-CONNECT.

10. ai-gateway -> kernel: SIO_QUERY_WFP_CONNECTION_REDIRECT_RECORDS

NOT IMPLEMENTED

Caller: would be in the accept/sniff path above

Lands in: nothing -- confirmed by direct search, zero hits for WFP_CONNECTION_REDIRECT_RECORDS / SIO_QUERY / SIO_SET anywhere under ai-gateway/. Explicitly an open item in the design doc section 5.

Before the call: N/A today.

After the call: sniff_inner / read_connect_authority currently get the destination from an explicit CONNECT line or SNI/Host -- a different mechanism, not the transparent-redirect recovery this step depicts.

11. ai-gateway (self): terminate TLS, relay through Optimus — VERIFIED

Caller: forward / forward_inner -- ai-gateway/listener/src/kinds/mod.rs:174, called from the service_fn closure https::serve builds

Lands in: same

Before the call: sniff_inner determined the connection shape and produced the CONNECT authority (if any).

After the call: forward_inner resolves the upstream (destination.resolve) and proxies the request onward.

12. ai-gateway -> kernel: own outbound connect() + SIO_SET_WFP_CONNECTION_REDIRECT_RECORDS — PARTIAL

Caller: forward_inner opens its own outbound connection (real)

Lands in: the SIO_SET stamping half is not implemented anywhere (same search as step 10; design doc section 5(c))

Before the call: the outbound connect itself is real.

After the call: the stamping isn't -- and hasn't bitten yet because today's filter conditions are scoped by ALE_APP_ID to VS Code's exe path specifically, so ai-gateway's own outbound connect (a different process) wouldn't match the filter anyway.

13. Kernel (self): ConnectRedirectClassify 2nd time -- loop guard fires — VERIFIED

Caller: same function again

Lands in: same function again

Before the call: per the note on step 12, this would only fire if a future, less-scoped filter made ai-gateway's own outbound connect match.

After the call: FwpsQueryConnectionRedirectState0 would report REDIRECTED_BY_SELF, and the function returns PERMIT unchanged -- no second redirect.

14. Kernel -> real CAPI host: connect() proceeds — VERIFIED

Caller: the OS TCP stack, permitted unchanged

Lands in: external -- api.githubcopilot.com's own server

Before the call: step 11/12's outbound connect initiated.

After the call: real TLS handshake with the actual host proceeds.

15. Real CAPI host -> ai-gateway: TLS handshake + model response — VERIFIED

Caller: external host

Lands in: forward_inner's response handling, ai-gateway/listener/src/kinds/mod.rs

Before the call: step 14.

After the call: forward_inner returns a Response<RelayBody> up through forward.

16. ai-gateway -> VS Code Copilot: relay response — VERIFIED

Caller: the response from step 15, written back over the original socket accepted in step 9

Lands in: VS Code Copilot's own TLS/HTTP client (external)

Before the call: step 15.

After the call: VS Code Copilot receives its model response -- believing, from its own TLS session (terminated by ai-gateway), that it spoke directly to api.githubcopilot.com.

Tally: 9 of 16 steps are real, verified code paths (1, 2, 5, 6, 7, 8, 9, 11, 15/16). One (3) is two real processes connected only by a file handoff. Two (4, 10) plus half of a third (12's SIO_SET half) are confirmed not implemented. One (13) is correct code guarding a case that cannot currently trigger, given today's narrow filter scoping.

The one blocker that matters most: step 4 (ListenerMapping / set_listener_mappings) gates the entire daemon-driven pipeline shut. Nothing downstream can produce an observable redirect without it, no matter how correctly everything else is configured -- though it does not block validating the kernel mechanism in isolation via a standalone filter-add test tool against a throwaway listener.